MIRAI BANK PROJECT

Milestone 2 [A2]

Data Storage Functional Design Document

Table of Contents

0. Scope of Implementation in the Milestone 3 Prototype 3
   0.1 Response to the Milestone 1 Review Comments 6
   0.2 Comparison of Prototype and Production Implementations 7
1. Purpose of This Document 10
2. Overall Storage Configuration 12
   2.1 What Is Stored and Where 12
   2.2 IPFS (Prototype) 13
   2.3 Arweave (Production) 14
   2.4 Data Structure and Size Limits 15
   2.5 Pinning Operations and Costs 16
   2.6 Arweave Migration Policy 21
3. Encryption and Decryption Design and Key Management Method (Corresponding to: A2-1) 23
   3.1 Design Policy 23
   3.2 Key Hierarchy (Production: Three Layers) 24
   3.3 Encryption Procedure (At Registration) 25
   3.4 Decryption Procedure (At Opening) 26
   3.5 Key Management Operations 28
   3.6 Data Model (Keys and Storage) 29
4. Redundancy and Failover Strategy (Corresponding to: A2-2) 31
   4.1 Basic Policy for Redundancy 31
   4.2 Failover Procedure on Retrieval 32
   4.3 Failure Scenarios and Responses 33
   4.4 Backups and Data Retention 34
5. Display Criteria for Stored Data and Search Mechanisms (Corresponding to: A2-3) 36
   5.1 Display Criteria by State 36
   5.2 Search Mechanisms 37
   5.3 Order of Retrieval for Display 38
   5.4 Rebuilding Without the Database (Re-Indexing) 39
6. API Design (Storage and Keys) 40
7. Key Design Decisions and Their Rationale 42
8. Summary 44

0. Scope of Implementation in the Milestone 3 Prototype

This document distinguishes two stages of implementation using the terms "prototype" and "production." The prototype refers to the system developed in Milestone 3. It runs on the Cardano Preprod testnet, and the opening decision is made by the server (Node.js) without using Aiken. Production refers to the production environment built in the full development that follows the prototype. It runs on Cardano Mainnet and introduces Aiken validators. Throughout this document, wherever the implementation differs between the two, the text states "prototype" or "production" explicitly, and Section 0.2 summarizes the overall comparison.

For the prototype, the scope of the user interface has been narrowed to a PC browser, individual (single-user) capsules only, one piece per capsule (a text, an image, or a PDF; images and PDFs up to 1 MB), and opening mode ① only. This chapter clarifies which parts of the storage and encryption design defined in this document (A2) are actually implemented in the Milestone 3 prototype.

This chapter does not change the data design or the key management design itself. The encryption method (AES-256-GCM, Section 3.1) and the approach of generating a random content key (CEK) and storing it wrapped are common regardless of content type or opening mode, and are applied to the prototype as is. The three-layer key hierarchy (Section 3.2) and key transfer protected by ECDH public keys (Sections 3.3 and 3.4) are retained as the production design; the prototype simplifies them to a two-layer key hierarchy and transfer over TLS (Table 0-1). Redundancy and failover (Chapter 4) are

retained as a design for the production environment, and only part of it is implemented in the prototype (Table 0-1).

Design Item	Treatment in the Milestone 3 Prototype	Section
Confirming holding of a master certificate (decryption condition)	Individual capsules are only ever decrypted by the registrant, so the branches for modes ② and ③ (approval count and quorum check) do not occur	3.4, Table 3-3, step 3
Content types stored (content_type)	One item per capsule: text (up to 100 KB), an image (JPEG or PNG, up to 1 MB), or a PDF (FILE, up to 1 MB). The format is checked on the device using the file type (MIME) and the leading bytes (magic number). Images are not resized. The server accepts requests of up to 2 MB	2.2, 2.4
Escrow distribution (service discontinuation)	Retained in the design as a production option. Out of scope for the prototype	3.5, Table 3-4
Search mechanisms (Section 5.2)	Each individual sees only their own capsules, so complex filtering and cross-group search are not used	5.2
IPFS pinning	Operated with a single Blockfrost IPFS service. The free Starter plan is used by default; the Hobby plan is used if a second project cannot be created (Section 2.5.3). Pin state management and operating rules follow Section 2.5 (the weekly export of the pin list is implemented in production)	2.2, 2.5
Redundancy and failover (implemented in the prototype)	Only content_hash verification on retrieval, sequential switching between two gateways (Blockfrost and ipfs.io), and	4.1, 4.2, 4.4

Design Item	Treatment in the Milestone 3 Prototype	Section
	the managed database's standard backups	
Redundancy and failover (implemented in production)	Scheduled health checks and automatic re-pinning, pinning with two providers, switching to a second blockchain API provider (Koios or Maestro), snapshots to another region, weekly export of the pin list, escrow distribution, and an offline decryption tool. The design is retained as described in Chapter 4 and Section 3.5	3.5, 4.1 to 4.4
Key hierarchy	Two layers: the CEK is wrapped directly with a single server key (MK, held in an environment variable). Per-capsule KEKs, KEK revocation (crypto-shredding), and MK rotation are implemented in production	3.2, 3.5
Transfer of the CEK	Over an encrypted connection (TLS) only. Double protection with ECDH public keys (the public wrapping key at registration and re-wrapping with an ephemeral public key at opening) is implemented in production	3.3, 3.4
Manifest JSON	Not created (unnecessary with one piece per capsule). Only the ciphertext is stored	2.1, 5.3, 5.4
Managing storage locations	A single location (URI) is recorded on the piece. Managing multiple paths with StorageObject is implemented in production	3.6
CID format	Recorded in the format returned by Blockfrost IPFS (no conversion to CIDv1)	2.2

Design Item	Treatment in the Milestone 3 Prototype	Section
Operational functions	No rate limit on key release, no split custody of the MK, and no automatic capacity monitoring. Capacity is checked visually on the Blockfrost dashboard	2.5.2, 3.5

Table 0-1 Scope of Implementation in the Milestone 3 Prototype

Chapters 1 through 8 describe the full design, including the items marked above as out of scope, together with the responses to the Milestone 1 review comments (Sections 0.1, 2.5, 2.6, and others).

0.1 Response to the Milestone 1 Review Comments

The Milestone 1 outputs were approved by the Catalyst reviewers in September 2026. At that time, the reviewers raised two points to be addressed in Milestone 2. This document responds to the second point: IPFS pinning operations and the Arweave migration policy. Table 0-2 shows where each part of the comment is addressed. The first point (where the fields used for the opening decision are held in the prototype, and the trust assumptions) is addressed in Output A1 (NFT Functional Design Document).

Review Comment	Response	Reference
Confirming IPFS pinning operations	Pinning was unified on Blockfrost IPFS, and operating rules were defined, including the correspondence between pin states and service-side states, the state transitions, and a prohibition on unpinning sealed data	Section 2.2, Sections 2.5.1 and 2.5.2
Confirming the cost of IPFS pinning	The Blockfrost plan structure was reviewed, and the free Starter plan was adopted as the	Section 2.5.3, Tables 2-7 and 2-8

Review Comment	Response	Reference
	default for the prototype. The Hobby plan (EUR 29 per month excluding tax) is used only if the two required projects cannot be created on Starter. The estimates confirm that even if every item is a 1 MB image or PDF, about 70 capsules fit on Starter, which is within the scale of prototype evaluation	
Notes on the Arweave migration policy	The unit of migration (switching the storage adapter), the timing of the switch, how to deal with the fact that location information in CIP-25 cannot be rewritten, the sealing conditions, costs, the role of IPFS, and the items to verify at migration were organized	Section 2.6, Table 2-9

Table 0-2 Milestone 1 Review Comments and Responses in This Document

0.2 Comparison of Prototype and Production Implementations

Table 0-3 summarizes the differences between the prototype and production implementations of the storage and encryption functions defined in this document. The encryption method (AES-256-GCM) is common to both. The main differences lie in the storage destination, the key hierarchy and where keys are kept, and the extent of redundancy.

Item	Prototype (Milestone 3)	Production
Content storage	A single Blockfrost IPFS service	Arweave (primary) plus IPFS mirrors (pinned with two providers)
Blockfrost plan	Free Starter plan by default; the	Selected according to

Item	Prototype (Milestone 3)	Production
	Hobby plan if a second project cannot be created (Section 2.5.3)	production usage
Content types and size limits	One item: text (up to 100 KB), or an image (JPEG or PNG) or a PDF (up to 1 MB)	Text, images, and files, with limits by type and plan (Table 2-4)
Encryption method	AES-256-GCM (Web Crypto API)	Same
Key hierarchy	Two layers, CEK and MK (the CEK is wrapped directly with the MK)	Three layers, CEK, KEK, and MK, supporting KEK revocation and MK rotation
Transfer of the CEK	Encrypted connection (TLS) only	TLS plus double protection with ECDH public keys
Manifest JSON	Not created	Created for each capsule and used for redundancy of location information
Where the master key (MK) is kept	Environment variable (Railway Secrets)	Inside the HSM of a cloud KMS; not exportable
Redundancy and failover	Only content_hash verification and sequential switching between two gateways (Blockfrost and ipfs.io)	Redundancy at three levels: storage, retrieval paths, and location information (Chapter 4)
Health checks and re-pinning	Not performed	Reachability checked every 24 hours, with re-pinning when unreachable
Redundancy of chain queries	Blockfrost only	Switching to a second provider (Koios or Maestro)
Database backups	Standard backups of the managed database	Daily snapshots to another region (retained for one year)

Item	Prototype (Milestone 3)	Production
Preparing for service discontinuation	Not performed	Escrow distribution of KEKs and an offline decryption tool
Search	Database search only (own capsules)	Three paths: database, on-chain, and storage side (Section 5.2)
Handling of data	Unpinned in bulk after evaluation ends (not migrated to Mainnet)	Stored permanently on Arweave

Table 0-3 Comparison of Prototype and Production Implementations (Storage and Encryption)

1. Purpose of This Document

This document is Milestone 2 ("Prototype detailed design") Output A2, "Prototype functional design: Data storage functional design document." Building on the decentralized storage policy selected in Milestone 1 (IPFS for the prototype, Arweave for production; Milestone 1 Output A1) and on client-side AES-256 encryption, it finalizes where and how encrypted content is stored, under which conditions it is retrieved and decrypted, and how the service continues when failures occur, at a level of detail that allows implementation in Milestone 3.

The acceptance criterion is a data storage design pattern, which must include at least the following three items for the stored data. Chapters 3, 4, and 5 correspond to these items one to one.

- **A2-1:**

  Encryption and decryption design and key management method for stored data (Chapter 3)

- **A2-2:**

  Redundancy and failover strategy for stored data (Chapter 4)

- **A2-3:**

  Display criteria for stored data and search mechanisms (Chapter 5)

The definitions of the location information, hashes, and encryption information recorded in the NFT are covered in Output A1 (NFT Functional Design Document), Section 3.3, and on-screen display and operations in Output B1 (User Interface Functional Design

Document). This document defines the storage, encryption, and retrieval mechanisms behind them. Field names (content_uri, content_hash, enc.*, and so on) are shared with A1.

2. Overall Storage Configuration

2.1 What Is Stored and Where

Table 2-1 shows the five kinds of data stored by this service. The content itself is always encrypted and placed in decentralized storage; the server database holds only location and search information. Because of this separation, the content does not leak even if the server is compromised, and conversely, the content can still be reached from the NFT and decentralized storage even if the server disappears.

Data	Encrypted	Prototype	Production	Notes
Encrypted content (text, images, files)	Yes	IPFS (Blockfrost IPFS)	Arweave (primary) plus IPFS (mirror)	One object per piece. Limits in Section 2.4
Manifest JSON (piece list and hashes)	No (public)	Not created	Arweave (primary) plus IPFS (mirror)	Contains no content. Output A1, Section 5.1
NFT image (thumbnail)	No (public)	IPFS (fixed CID)	Arweave plus IPFS	One image common to the service
NFT metadata	No (public)	Cardano Preprod	Cardano Mainnet	CIP-25. Output A1, Chapter 5
Metadata, key references, search indexes	Partly (keys)	PostgreSQL or MySQL (Prisma)	Same (managed)	Deletable; supports deletion requests such as under the GDPR

Table 2-1 What Is Stored and Where

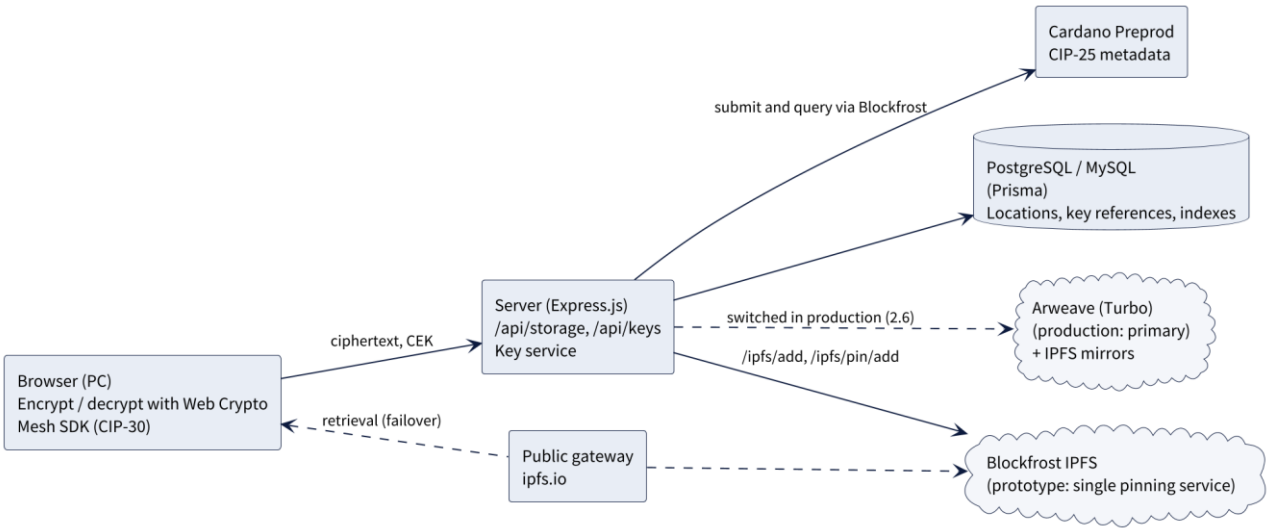


Figure 2-1 Overall Storage Configuration

2.2 IPFS (Prototype)

Item	Design
Pinning service	**Blockfrost IPFS is adopted. NFT.Storage had originally been considered, but it ended its free uploads (Classic) in 2024. Blockfrost IPFS is provided by the same company, with the same authentication method (project_id header), as the Cardano node access, so it can be operated with settings shared with src/services/cardano.js. A project_id for IPFS must be issued separately from the one for Cardano Preprod (Section 2.5.3). The implementation is an adapter layer, src/services/ipfs.js, so that it can be replaced with another pinning service.**
CID format	**Prototype: the CID returned by Blockfrost IPFS is recorded as is (even the CIDv0 format fits the 64-byte limit of the metadata). Production: converted to CIDv1 (base32) before recording. The two are equivalent representations of the same content, and conversion does not change the content.**
Upload path	**The ciphertext encrypted in the browser is sent through the server (/api/storage/upload) to Blockfrost's /ipfs/add, followed by a pinning request to /ipfs/pin/add/{CID}. It is not sent directly from the browser, to protect the project_id.**
Retrieval paths	**Multiple paths, including public gateways, are registered and tried in order**

Item	Design
	(Chapter 4). Prototype default: the Blockfrost IPFS gateway (/ipfs/gateway/{CID}, through the server), then ipfs.io (two paths). Production adds dweb.link for three or more paths. cloudflare-ipfs.com, which ended its service in August 2024, is not used.
Pin confirmation	The pin state is checked with /ipfs/pin/list/{CID}, and the "temporarily saved" state of a DRAFT is not confirmed until the state becomes pinned (see Section 2.5.1 for the correspondence of states).

Table 2-2 IPFS Design (Prototype)

2.3 Arweave (Production)

Item	Design
Upload method	Through Turbo (a bundler). Small uploads (up to about 100 KiB in total) fall within the free tier; beyond that, credits are used (non-ADA payments are managed by operations).
Identifier	ar://{TxID}. The TxID is 43 base64url characters and is recorded as a long_string in the NFT metadata.
Tags	Content-Type: application/octet-stream, App-Name: MIRAI-BANK, Capsule-Id, Piece-Id, and Content-Hash are added as Arweave tags so that the data can be searched with Arweave GraphQL (Chapter 5).
Mirror	In production, the same ciphertext is also pinned on IPFS and recorded in the manifest's mirror_uris, as an alternative path when Arweave gateways fail (Chapter 4).
Waiting for confirmation	Turbo immediately returns an optimistic receipt ID, but until inclusion in a block is confirmed, the database holds storageStatus = PENDING, which is updated to CONFIRMED after confirmation. Sealing (NFT minting) requires CONFIRMED.

Table 2-3 Arweave Design (Production)

2.4 Data Structure and Size Limits

A stored object is a single byte sequence consisting of a fixed-length header followed by the ciphertext. The header allows the method and the IV to be recovered even without the NFT metadata or the database, and accommodates future changes of method.

```
Stored object (byte sequence)
 [0..3]   magic     "MBTC" (4 bytes)
 [4]      version   0x01
 [5]      alg       0x01 = AES-256-GCM
 [6..17]  iv        12 bytes (same value as enc.iv)
 [18..19] ct_type   0x0001 TEXT / 0x0002 IMAGE / 0x0003 FILE
 [20..]   ciphertext  plaintext + authentication tag (16 bytes)

Plaintext structure (before encryption, JSON encoded as UTF-8)
 { "v":1, "type":"TEXT|IMAGE|FILE", "name":"...", "mime":"...", "created_at":...,
"data":"<base64>" }
```

Figure 2-2 Structure of a Stored Object

Type	Prototype Limit	Production Limit (Initial Plan)	Notes
TEXT	100 KB	1 MB	Body in UTF-8
IMAGE	1 MB	20 MB	JPEG and PNG in the prototype. Production also accepts WebP and offers the option of resizing the long side to 2048 px on the device
FILE	1 MB	50 MB	PDF only in the prototype; PDF and similar in production. Video is out of scope
Capsule total	One piece (max. 1 MB)	By plan	In production, linked to the service plan design

Table 2-4 Size Limits

2.5 Pinning Operations and Costs

On IPFS, data that is not pinned is removed by periodic garbage collection. This section defines, for Blockfrost IPFS used in the prototype, the management of pin states, the operating rules, and the estimates of cost and capacity.

2.5.1 Pin States and the Database

Blockfrost IPFS reports the state of a pinned object as one of five values: queued, pinned, failed, unpinned, or gc. The server records this state (in the prototype, on the piece record; in production, in StorageObject.pinState) and maps it to the service-side state as shown in Table 2-5. Figure 2-3 shows the state transitions.

Blockfrost State	Meaning	Service-Side State	Action by the Service
queued	Waiting to be pinned	PENDING	The temporary save is not yet confirmed. /ipfs/pin/list/{CID} is queried again every 20 seconds
pinned	Pinning completed	CONFIRMED	The temporary save is confirmed. A precondition for sealing (NFT minting)
failed	Retrieval failed	PENDING (retry)	/ipfs/pin/add is requested again. After three failures, the user is asked to register again
unpinned	Unpinned (subject to removal)	Never allowed for sealed data	Occurs only when a DRAFT is replaced or discarded
gc	Removed	UNREACHABLE	If detected for sealed data, treated

Blockfrost State	Meaning	Service-Side State	Action by the Service
			as a failure and re-pinning is attempted (Section 4.3)

Table 2-5 Pin States and Actions by the Service

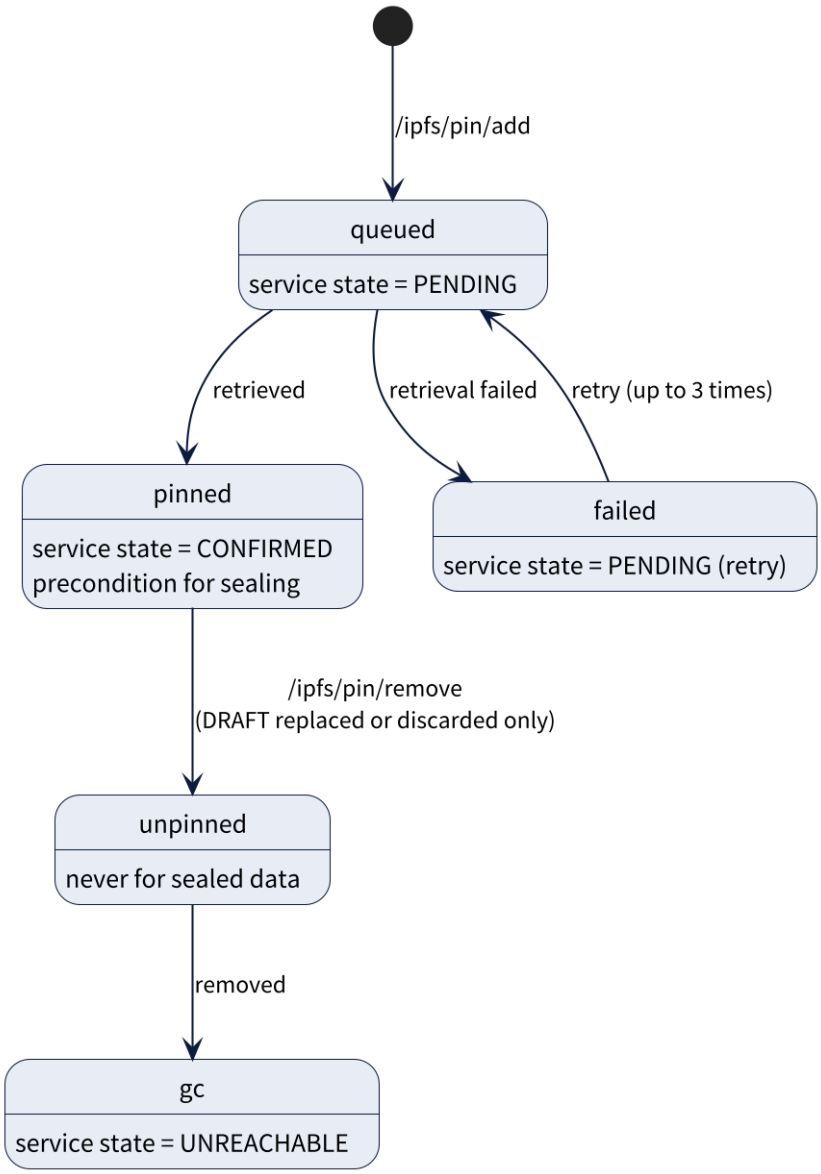


Figure 2-3 Pin State Transitions

2.5.2 Operating Rules

Item	Rule
No unpinning of sealed data	The ciphertext and manifest of sealed (LOCKED or OPENED) capsules, and the NFT image, are never unpinned. Unpinning can be

Item	Rule
	executed only from the server's administrative function, and requests for sealed CIDs are rejected.
Replacement during DRAFT	When the temporarily saved content is replaced, the old CID is unpinned after the replacement is confirmed (to save capacity).
Discarding a DRAFT	The CIDs of a capsule discarded without being sealed are unpinned when the discard is confirmed.
Capacity monitoring	Production: the size values from /ipfs/pin/list are totaled daily, and operators are notified when 80% of the plan capacity is reached. Prototype: checked visually on the Blockfrost dashboard (usage is very small relative to Starter's 100 MB).
Limit on pending pins	On Blockfrost, a user can have at most 100 pins in the queued state. When exceeded (HTTP 425), the service waits and retries. The prototype stores only one ciphertext per capsule, so the limit is not normally reached.
Filecoin long-term storage option	Blockfrost IPFS can also store data on the Filecoin network when filecoin=true is added to a pinning request. However, data stored on Filecoin cannot be unpinned later. Because prototype data is to be discarded after evaluation, this option is not used in the prototype. It is a candidate when selecting the second IPFS mirror provider for production (Section 2.6).
Backup of the pin list	The list of pinned CIDs is exported to the database weekly (Section 4.4). Implemented in production; not performed in the prototype.
After the prototype period	Prototype data on the testnet is not migrated to production (Section 2.6). After evaluation of the prototype ends, all data except capsules kept for evaluation is unpinned in bulk.

Table 2-6 Operating Rules for Pinning

2.5.3 Estimates of Cost and Capacity

On Blockfrost, each project (the unit for which an API key, the project_id, is issued) can be used with only one network. The prototype uses both Cardano Preprod (issuing certificates, confirming holdings, and reading metadata) and IPFS (storing and retrieving ciphertext), so one account needs two projects, one for Preprod and one for IPFS. Plan limits such as the number of requests and IPFS capacity are shared by the two projects.

The prototype uses the free Starter plan by default. However, the pricing page shows one project for Starter, so it may not be possible to create a second project. In the first sprint of Milestone 3, a second project will be created from the dashboard; if this is not possible, the Hobby plan will be used. Table 2-7 shows the main plans (as of September 2026; prices exclude tax).

Plan	Position in the Prototype	Monthly Fee	Requests per Day	IPFS Capacity	Projects (as Shown)
Starter	Default	Free	50,000	100 MB	1
Hobby	Adopted if a second project cannot be created on Starter	EUR 29	300,000	1,000 MB	3
Developer	Not used in the prototype	EUR 79	1,000,000	50,000 MB	10

Table 2-7 Main Blockfrost Plans and Their Position in the Prototype

Table 2-8 estimates whether prototype usage fits within the Starter limits; if it fits within Starter, it naturally fits within Hobby as well. Each prototype capsule holds one piece: a text (up to 100 KB), an image (up to 1 MB), or a PDF (up to 1 MB) (Table 2-4). Formatting as

plaintext JSON (base64 encoding) increases the size by about 1.33 times, to which the 20-byte fixed-length header and the 16-byte authentication tag are added.

Item	Estimate (Starter: IPFS 100 MB, 50,000 Requests per Day)
Storage per capsule (maximum)	About 1.34 MB of ciphertext for a 1 MB image or PDF (the prototype does not create a manifest). Up to about 136 KB for text, and less than 10 KB for typical text of a few KB
Common NFT image	Only one for all capsules (kept within 200 KB)
Number of capsules that fit in 100 MB	About 70 capsules if all are 1 MB images or PDFs, and several hundred if text or images of a few hundred KB predominate. Enough for prototype evaluation (a few dozen capsules). For larger evaluations, the Hobby plan is used (1,000 MB; about 700 capsules at the maximum size)
Requests to seal one capsule	About 5 for IPFS (add, pin, and state queries) and about 10 to 20 for Cardano (UTxO queries, transaction submission, and confirmation of inclusion)
Requests for one opening	2 to 3 for Cardano (holding check and metadata retrieval) and 1 for IPFS (retrieval)
Requests per day	Even sealing and opening 100 capsules per day takes about 3,000 requests, small relative to the limit of 50,000. The prototype does not perform health checks (Table 0-1), so no requests arise from scheduled jobs
Cost during the prototype period	Free on Starter. On Hobby, EUR 29 per month × the prototype period (about EUR 58 excluding tax for roughly eight weeks)

Table 2-8 Estimates of Capacity and Requests on Starter

These costs are operating costs of the service and are not charged to prototype users in ADA (Output A1, Chapter 7).

2.6 Arweave Migration Policy

The policy of using Arweave as the primary storage in production is maintained. This section organizes, as notes, the points that should be decided now in preparation for the migration; the detailed design will be done when development for production begins.

Topic	Policy
Unit of migration	The migration is carried out as a switch of the storage adapter (src/services/ipfs.js to arweave.js), not as a transfer of stored data. Prototype capsules are tied to certificates on Preprod and are not migrated to Mainnet, so no work arises to move prototype data to Arweave.
Timing of the switch	When moving to production (Mainnet), Arweave becomes the primary storage for newly sealed capsules. The switch is made in the same release that introduces Aiken validators, so that no capsule on Mainnet ever has IPFS as its primary storage.
Location information cannot be rewritten	CIP-25 metadata cannot be changed after sealing, so the NFT's content_uri cannot later be rewritten to ar://. In production, ar:// is recorded in content_uri at sealing, and the IPFS CID is recorded in the manifest's mirror_uris (Section 2.3). The design does not assume any future transfer.
Sealing conditions	Both confirmation of inclusion on Arweave (storageStatus = CONFIRMED) and the pinned state of the IPFS mirror are preconditions for sealing.
Cost estimates	Calculated from the byte count using Turbo's pricing API. For the initial text-centered plan, the cost per capsule is small; as images and files are enabled, costs are linked to the storage limits of each plan.
Role of IPFS	In production, IPFS remains as a mirror for redundancy of retrieval paths. Pinning uses two providers; the provider added

Topic	Policy
	to Blockfrost IPFS (or Blockfrost IPFS's Filecoin long-term storage option) will be selected when development for production begins, comparing cost and permanence (Table 4-1).
Items to verify at migration	Replacement of the storage adapter, additional values of StorageObject.provider, Arweave tags (Section 2.3), GraphQL search (Section 5.2), and the re-indexing procedure (Section 5.4), all verified in the production environment.

Table 2-9 Arweave Migration Policy

3. Encryption and Decryption Design and Key Management Method (Corresponding to: A2-1)

3.1 Design Policy

- **Encryption on the device:**

  Content is encrypted in the user's device (browser), and plaintext is never sent to the server. Plaintext exists in none of the server, the storage, or the blockchain.

- **AES-256-GCM with Web Crypto:**

  The encryption method is AES-256-GCM, implemented with the browser-standard Web Crypto API (SubtleCrypto). As authenticated encryption, it automatically detects tampering during decryption.

- **No keys derived from the opening date:**

  Keys are not derived from the opening date/time. The opening date/time is public information, and a derived key could be computed by anyone. Keys are randomly generated for each piece and stored wrapped.

- **Release only to the eligible person, each time:**

  A key is released only to the person who meets the decryption conditions (Output A1, Section 3.4), each time it is needed. The released key is used only in the device's memory and is never written to disk.

- **The service is a custodian, not a reader:**

  The service keeps the keys but does not read the content. No path is provided through which the key service can access the ciphertext (separation of privileges).

3.2 Key Hierarchy (Production: Three Layers)

Layer	Key	Generation and Storage	Role
1	CEK (Content Encryption Key), 256 bits, per piece	Randomly generated on the device (crypto.getRandomValues). After encrypting the plaintext, it is wrapped with the KEK, sent to the server, and discarded on the device.	Encrypts the content. Referenced from enc.key_ref in the NFT.
2	KEK (Key Encryption Key), 256 bits, per capsule	Generated by the server's key service when the capsule is created; encrypted with the MK and stored in the database (CapsuleKey).	Wraps all CEKs of the capsule. Enables key revocation and rotation per capsule.
3	MK (Master Key), 256 bits, per environment	Generated inside the HSM of a cloud KMS (AWS KMS or GCP Cloud KMS) and never taken outside. (Prototype: environment variable, Railway Secrets.)	Encrypts KEKs. The KMS permits only wrap and unwrap operations, and the key itself cannot be exported.

Table 3-1 Three-Layer Key Hierarchy

With this hierarchy, even if the database leaks, the CEKs cannot be recovered (because the KEKs are encrypted with the MK); conversely, obtaining only the MK does not give access to the content without both the wrapped keys in the database and the ciphertext. Moving from the prototype to production changes only where the MK is kept (from an environment variable to a KMS) and adds the KEK layer.

In the prototype, the key hierarchy is simplified to two layers, CEK and MK. The CEK is wrapped directly with AES-KW using a single server key (the MK, held in an environment variable) and stored in PieceKey.wrappedCek. No per-capsule KEK (CapsuleKey) is created. The encryption method (AES-256-GCM) and the approach of generating a random

CEK and storing it wrapped are the same as in production; production adds the KEK layer and moves to a KMS.

3.3 Encryption Procedure (At Registration)

#	Where	Processing
1	Device	Format the file or text selected by the user as plaintext JSON (Figure 2-2) and compute its SHA-256 → plain_hash
2	Device	Randomly generate the CEK (256 bits) and the IV (96 bits)
3	Device	Encrypt with AES-256-GCM (capsule_id and piece_id are given as AAD to prevent reuse of the ciphertext for another piece). Add the header to create the stored object and compute its SHA-256 → content_hash
4	Device → server	Send the stored object to /api/storage/upload and receive content_uri (CID) in the response
5	Device → server	Before sending the CEK to the server, encrypt it with the capsule's public wrapping key (an ECDH public key derived from the KEK) obtained from the server. The plaintext CEK is thus doubly protected even on the connection (within TLS)
6	Server (key service)	Wrap the received CEK with the KEK (AES-KW) and store it in the database as PieceKey (key_ref, wrapped_cek, iv, alg). Discard the plaintext CEK from memory
7	Server	Record content_uri, content_hash, plain_hash, and enc.* on the Piece and update its state to DRAFT (temporarily saved)
8	Device	Discard the CEK and the plaintext from memory. Nothing is stored in browser storage (localStorage, etc.)

Table 3-2 Encryption Procedure (At Registration)

In the prototype, the double protection with an ECDH public key in step 5 is not performed; the CEK is sent to the server over the encrypted connection (TLS). The server

wraps the CEK with the MK and stores it.

3.4 Decryption Procedure (At Opening)

Decryption takes place on the device, after the server confirms that the viewer meets the decryption conditions (C-1 to C-3 in Output A1, Table 3-7) and releases the CEK only to that viewer. Figure 3-1 shows the flow.

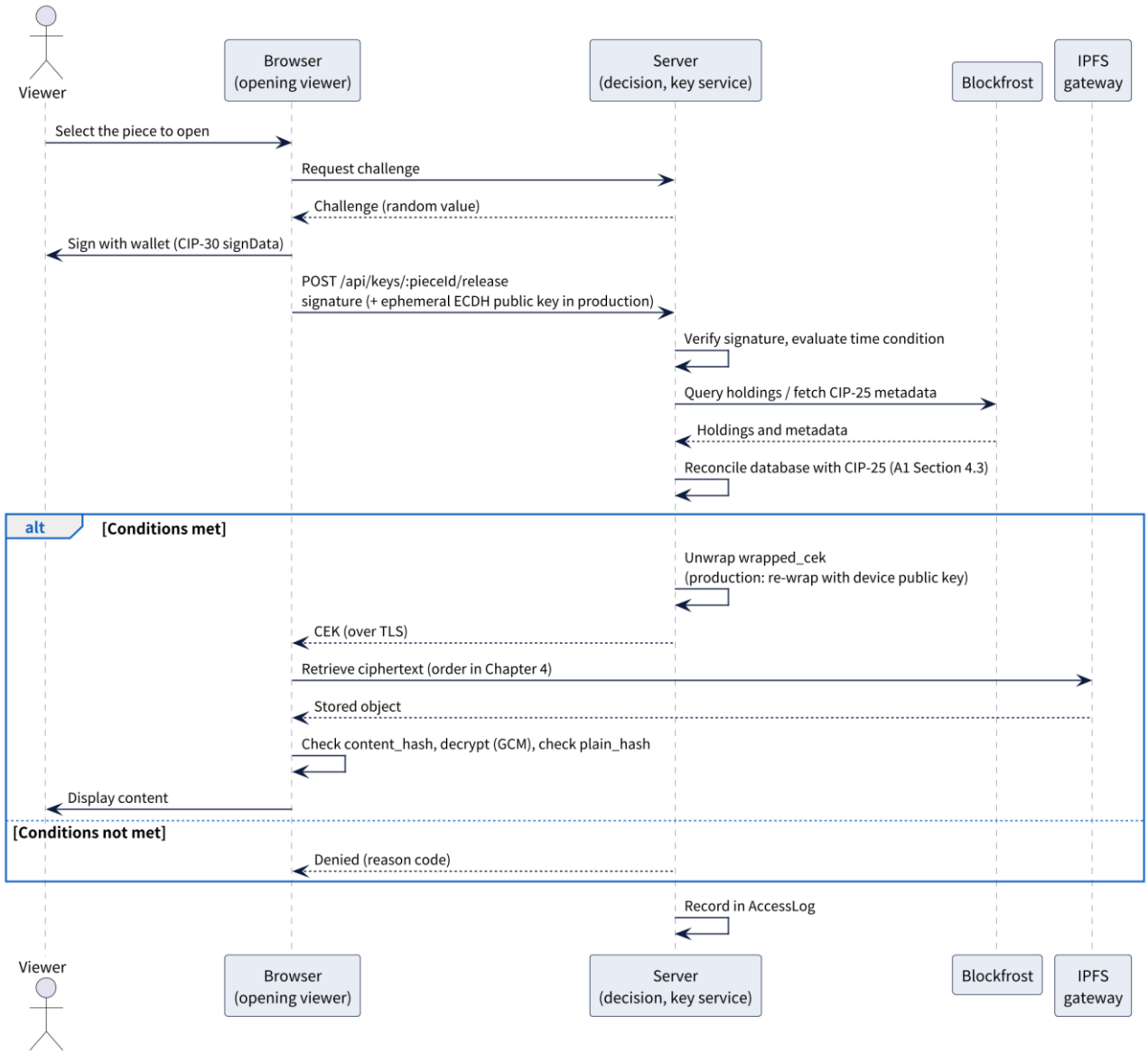


Figure 3-1 Key Release and Decryption at Opening

#	Where	Processing
1	Device	Select the piece to open on SCREEN-09 (Opening Viewer). Sign the

#	Where	Processing
		challenge (random value) received from the server with the wallet (CIP-30 signData)
2	Server	Verify the signature and determine the signer's address. Evaluate the time condition based on unlock_at and opening_mode
3	Server → Blockfrost	Query the assets held by the signer's address and confirm holding of the piece certificate or a master certificate (in mode ③, also confirm that the approval count ≥ quorum). Also retrieve the CIP-25 metadata of the asset and confirm that it matches the database values used for the decision (Output A1, Section 4.3). If they do not match, the key is not released
4	Server (key service)	Only if the conditions are met, unwrap wrapped_cek with the KEK, re-wrap the CEK with the ephemeral ECDH public key generated on the viewer's device, and return it (the plaintext CEK is never exposed on the connection from the server to the device)
5	Device	Retrieve the stored object from content_uri (including mirrors, in the order in Chapter 4) and confirm that its SHA-256 matches content_hash
6	Device	Unwrap the CEK and decrypt with AES-256-GCM. If verification of the GCM authentication tag fails, the content is not displayed and the error is reported as tampering or a key mismatch
7	Device	Confirm that the SHA-256 of the decrypted plaintext matches plain_hash, and display it according to content_type (text as the body, images on screen, PDFs in the browser PDF viewer with a download option)
8	Server	Record the opening log (who, when, and under which condition) in AccessLog. The content is not recorded

Table 3-3 Decryption Procedure (At Opening)

In the prototype, in step 4 the wrapped_cek is unwrapped with the MK, and the CEK is

returned over the encrypted connection (TLS). Re-wrapping with the device's ephemeral ECDH public key is implemented in production.

3.5 Key Management Operations

Operation	Design
Key backups	KEKs (encrypted with the MK) are included in the managed database's automatic backups (Neon: point-in-time recovery) as part of the database. In production, the MK relies on the redundancy of the KMS. In the prototype, the MK is held in an environment variable.
MK rotation	Once a year using the KMS's key rotation function. KEKs encrypted with the old MK are re-encrypted with the new MK the next time they are accessed (lazy rotation).
KEK revocation	Possible per capsule. When a group is dissolved or a user requests deletion, the KEK is destroyed, making the ciphertext undecryptable even if it remains in decentralized storage (crypto-shredding).
Rate limit on key release	Up to 5 key releases per minute for the same piece. Consecutive failures are recorded in the audit log.
Long-term storage risk	AES-256 is currently considered sufficient even against quantum computers expected beyond 2050, but because storage for 100 years is assumed, room is left to add post-quantum key encapsulation (ML-KEM) to the KEK layer (by extending the enc.wrap enumeration).
Service discontinuation	As a production option, the design includes "escrow distribution," in which KEKs are distributed to group managers using threshold secret sharing (Shamir, e.g., 3 of 5) so that content can be decrypted by the managers' agreement even if the service disappears.

Table 3-4 Key Management Operations

3.6 Data Model (Keys and Storage)

The following is the production data model. The prototype creates only PieceKey (wrappedCek, iv, alg, keyRef) and AccessLog; CapsuleKey (KEK) and StorageObject (multiple storage paths) are not created. The location is recorded once, in Piece.contentUri.

```
model CapsuleKey {          // KEK per capsule (encrypted with the MK)
  id          String  @id @default(cuid())
  projectId   String  @unique
  wrappedKek  Bytes           // KEK encrypted with the MK
  mkVersion   Int             // Version of the MK used
  wrapPubKey  Bytes           // ECDH public key protecting the CEK from device to server
  revokedAt   DateTime?       // Revocation time (crypto-shredding)
}

model PieceKey {            // CEK per piece (wrapped with the KEK; with the MK in the prototype)
  id          String  @id @default(cuid())
  pieceId     String  @unique
  keyRef      String  @unique      // Same as enc.key_ref  k1:{capsule}:{piece}
  wrappedCek  Bytes           // CEK wrapped with AES-KW
  iv          Bytes           // 96 bits
  alg         String  @default("AES-256-GCM")
}

model StorageObject {       // Locations of stored objects (multiple paths)
  id          String  @id @default(cuid())
  pieceId     String
  kind        StorageKind      // CONTENT | MANIFEST
  provider    String          // IPFS_BLOCKFROST | ARWEAVE_TURBO
  uri         String          // ipfs://... | ar://...
  isPrimary   Boolean
  status      StorageStatus    // PENDING | CONFIRMED | UNREACHABLE
  pinState    String?         // Blockfrost pin state (Section 2.5)
  sizeBytes   Int
```

```
  contentHash   String            // SHA-256 hex
  lastCheckedAt DateTime?
}

model AccessLog {           // Opening log (does not hold content)
  id       String   @id @default(cuid())
  pieceId   String
  wallet    String
  condition String           // C-1 | C-2 | C-3 | C-4
  result    String           // GRANTED | DENIED_TIME | DENIED_HOLDER
                             // | DENIED_QUORUM | DENIED_MISMATCH
  at       DateTime @default(now())
}
```

Figure 3-2 Prisma Models for Keys and Storage (Additions)

4. Redundancy and Failover Strategy (Corresponding to: A2-2)

This chapter is the redundancy and failover design for the production environment. The scope implemented in the Milestone 3 prototype is limited to hash verification on retrieval, sequential switching between gateways, and the managed database's standard backups; everything else is implemented in production (Chapter 0, Table 0-1).

4.1 Basic Policy for Redundancy

With the goal of being able to open capsules even 100 years from now, the configuration does not depend on a single provider, a single network, or a single path. Redundancy is provided at three levels: storage, retrieval paths, and location information.

Level	Means of Redundancy	Prototype	Production
Storage	The same ciphertext is placed in multiple stores. Because IPFS uses content addressing (CIDs), identity can be verified whichever path is used	A single IPFS service (Blockfrost IPFS)	Arweave (primary) plus IPFS (mirror, pinned with two providers: Blockfrost IPFS and one other)
Retrieval paths	Multiple gateways are registered for the same store and switched sequentially	Two IPFS gateways (Blockfrost and ipfs.io)	Two Arweave gateways plus three IPFS gateways
Location information	Locations are held in three places (database, NFT metadata, and manifest), so the content can be reached from the NFT through the manifest	Two places (database and NFT metadata; no manifest)	Three places

Level	Means of Redundancy	Prototype	Production
	even if the database is lost		
Integrity	Retrieved data is verified with content_hash; if it does not match, another path is used	Same	Same

Table 4-1 Levels of Redundancy

4.2 Failover Procedure on Retrieval

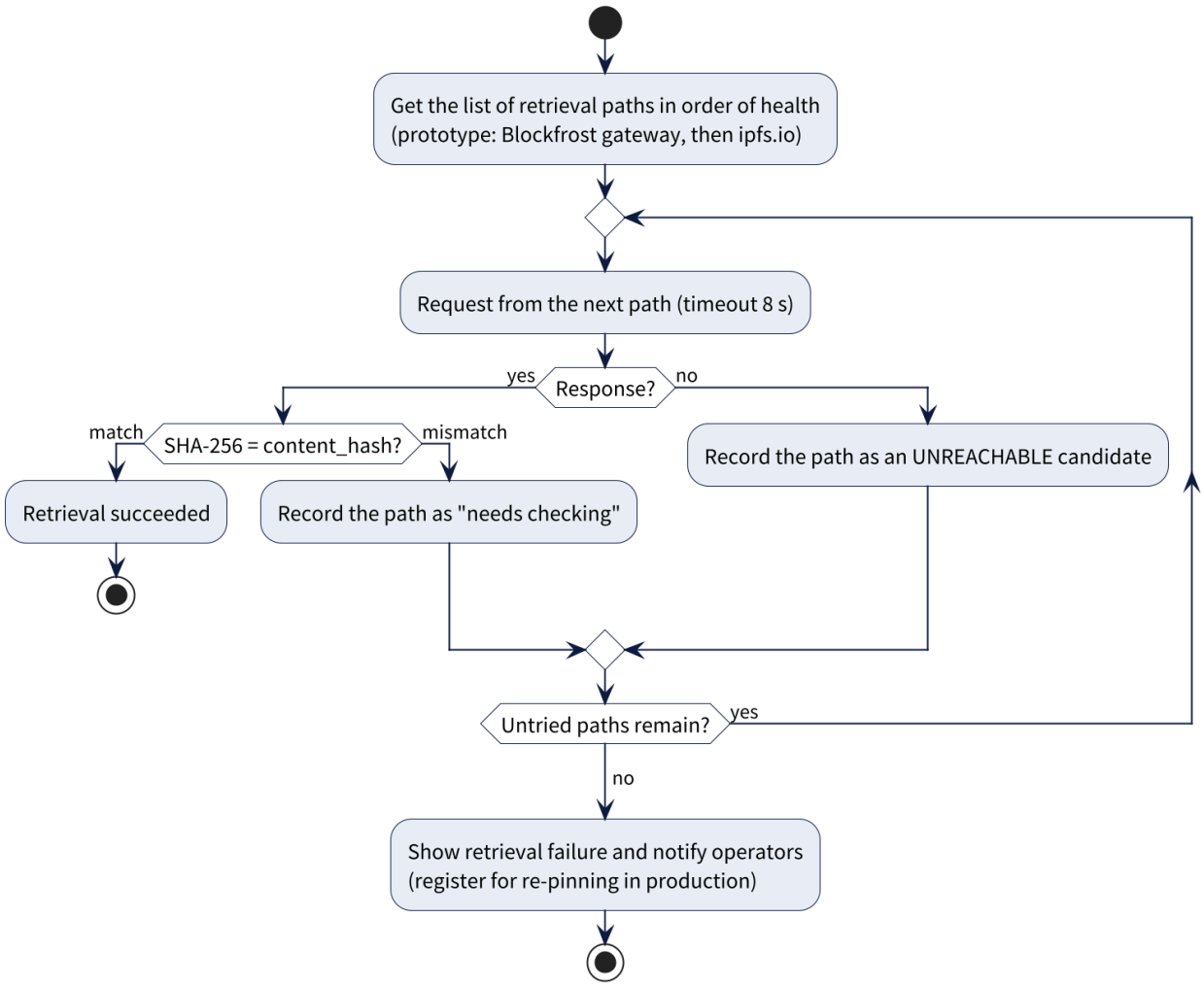


Figure 4-1 Failover on Retrieval

Parameter	Default	Description
Timeout per path	8 seconds	Moves to the next path when exceeded
Maximum number of	5 (2 in the	Two Arweave plus three IPFS gateways

Parameter	Default	Description
paths tried	prototype)	
Retries on hash mismatch	3	Another path is preferred
Health check interval (production)	24 hours	node-cron. Reachability of every StorageObject is checked, and UNREACHABLE is recorded
Re-pinning (production)	When UNREACHABLE is detected	A CID that has become unreachable on IPFS is retrieved from a mirror (another pinning service or Arweave) and pinned again

Table 4-2 Failover Parameters

4.3 Failure Scenarios and Responses

Scenario	Impact	Response
Outage or withdrawal of the pinning service (IPFS)	Content cannot be retrieved through that service's gateway; the CID remains valid	While the content can still be retrieved through other gateways or mirrors, it is pinned again with another service. The prototype has a single pinning service, so if the content cannot be retrieved through any path, it cannot be viewed until service is restored (a risk accepted at the prototype stage). In production, Arweave is primary, so viewing is not affected
Arweave gateway failure	Delay in retrieving ar:// content	Switch to the second gateway, then to the IPFS mirror. The data on Arweave itself is permanent
Loss of the server database	Search, lists, and the key	Restore from the managed

Scenario	Impact	Response
	service stop	database's point-in-time recovery. Keys are restored together with the encrypted KEKs. Locations can be rebuilt from the NFTs (Section 5.4, re-indexing)
No access to the MK (KMS)	No capsule can be decrypted (the ciphertext is safe)	Depends on the availability SLA of the KMS. Production uses multi-region keys. In the long term, mitigated by escrow distribution (Section 3.5)
Blockfrost failure	Holding of NFTs cannot be confirmed, so no opening decision can be made	Switch to a second provider (Koios or Maestro); cardano.js is implemented with a provider abstraction (production)
Discontinuation of the service itself	The key service stops	An "offline decryption tool" is published that can decrypt using only the escrowed KEKs and public information (NFTs, manifests, IPFS, and Arweave)

Table 4-3 Failure Scenarios and Responses

4.4 Backups and Data Retention

Target	Method	Retention Period
Server database	Automatic backups and point-in-time recovery of the managed database; daily snapshots to another region in production	Snapshots for 90 days; 1 year in production

Target	Method	Retention Period
Ciphertext (IPFS)	One provider (Blockfrost IPFS) in the prototype; two pinning providers in production, with a weekly export of the pin list to the database	The capsule's unlock_at plus 10 years (depends on the plan)
Ciphertext (Arweave)	Arweave itself provides permanent storage; no backup required	Permanent
Keys (KEK, wrapped CEK)	Included in the database. In addition, only the encrypted KEKs are exported monthly to separate media (production)	Same as the database, plus 1 year after revocation (for recovery from accidental deletion; then erased completely)

Table 4-4 Backups and Data Retention

5. Display Criteria for Stored Data and Search Mechanisms (Corresponding to: A2-3)

5.1 Display Criteria by State

What stored data is displayed, to whom, and when is determined by the state of the capsule and the viewer's position. The content is displayed only on the device of the person who meets the decryption conditions, in any state, and is never included in the server's screen output.

Information	DRAFT	LOCKED (Sealed, Before Opening)	OPENED	Third Party (Not Logged In)
Capsule name, description, opening date/time, opening mode	Members	Members	Members and delegated viewers	NFT holders only (public verification page)
Piece list (slot numbers, registration status)	Members (managers see all)	Members (own status); managers (all)	Same	Count only
Registrant's display name	Members	Members	Members	Shortened address only
content_uri, content_hash, plain_hash	Registrant and managers	Registrant and managers	Registrant, managers, and delegated viewers	Public (visible in the NFT metadata)
Content (after	Registrant only	Not displayed (except	Only on the	Not displayed

Information	DRAFT	LOCKED (Sealed, Before Opening)	OPENED	Third Party (Not Logged In)
decryption)	(their own temporarily saved content is redisplayed on the device)	emergency opening in mode ③)	devices of those who meet the decryption conditions	
On-chain verification (transaction, policy ID, hash match)	—	All members	All members and delegated viewers	Public
Opening log	—	Managers	Managers	—

Table 5-1 Display Criteria by State and Position

5.2 Search Mechanisms

Search has two paths: normal search using the server database, and recovery search from the blockchain and decentralized storage. The former is the everyday path; the latter is for when the server is unavailable or for third-party verification.

Path	Search Keys	Mechanism
Database search (normal)	Wallet address, group ID, capsule ID, state, range of opening dates, free text (capsule name and description)	Through Prisma. Indexes: GroupMembership(walletAddress), Project(groupId, status, unlockAt), Piece(projectId, slotNo). Free text uses the database's full-text search (PostgreSQL tsvector or MySQL FULLTEXT)
On-chain search	Policy ID, asset name, wallet	Blockfrost: list held assets with

Path	Search Keys	Mechanism
	address	/addresses/{addr}/utxos, then retrieve CIP-25 metadata with /assets/{asset} to obtain content_uri and manifest_uri. Locations can be reached even without the server database
Storage-side search	Arweave tags (App-Name, Capsule-Id, Piece-Id, Content-Hash)	Search with Arweave GraphQL for App-Name = MIRAI-BANK and Capsule-Id = ... to recover the TxID. IPFS uses content addressing, so the CID must already be known

Table 5-2 Search Paths

5.3 Order of Retrieval for Display

When displaying data on screen, data is retrieved in the following order to balance responsiveness and reliability.

#	Source	What Is Retrieved	How It Is Displayed
1	Server database	Metadata for lists and details (immediate)	Displayed first, with a "verifying on-chain" badge
2	Blockfrost	Existence of the NFT, its metadata, and the current holder	"Verified" if it matches the database values; "needs checking," with the differences, if not
3	Decentralized storage	Manifest, then comparison with content_hash	"Integrity confirmed" if they match. The content itself is retrieved only at the opening operation (no prefetching)

Table 5-3 Order of Retrieval for Display

For capsules that have not yet been opened, the ciphertext is not prefetched. Retrieving it

would do no harm, but to reduce network cost and gateway load, it is retrieved for the first time at the opening operation (SCREEN-09, Opening Viewer).

In the prototype, no manifest is created, so step 3 retrieves the ciphertext from content_uri in the NFT metadata and compares it with content_hash. List screens go only as far as step 2, and step 3 is performed only on SCREEN-09 (Opening Viewer).

5.4 Rebuilding Without the Database (Re-Indexing)

This section defines the procedure for rebuilding the list of capsules from public information alone, when the server database is lost or when a third party verifies independently. The fact that this procedure works is what supports the design goal that capsules can be opened even after the service is gone.

1. List the assets held by the target wallet with Blockfrost and extract those whose asset names begin with "MB-"
2. Retrieve the CIP-25 metadata (721) of each asset -> obtain capsule_id, kind, manifest_uri, unlock_at, enc
3. Retrieve the manifest from manifest_uri and compare it with manifest_hash (database or Datum)
4. Recover content_uri, mirror_uris, and content_hash of all pieces from pieces[] in the manifest
5. Regenerate Project, Piece, and StorageObject (display names and descriptions supplemented from description)
6. Restore keys (PieceKey, CapsuleKey) from backups, or re-register them from the escrowed shares

Figure 5-1 Re-Indexing Procedure

This procedure applies to production. The prototype does not create a manifest, so instead of steps 3 and 4, the location of each piece is recovered directly from content_uri and content_hash in the NFT metadata (sufficient because each capsule has one piece).

6. API Design (Storage and Keys)

APIs whose description begins with "(Production only)" are not implemented in the prototype.

Method	Path	Description	Authentication
POST	/api/storage/upload	Receives a stored object and forwards it to Blockfrost IPFS (/ipfs/add and /ipfs/pin/add) or Turbo. Returns content_uri and sizeBytes	wallet
GET	/api/storage/resolve/:pieceId	(Production only) Returns the list of retrieval paths for a piece (primary plus mirrors, in order of health)	wallet
GET	/api/storage/health/:pieceId	(Production only) Reachability of each path (CONFIRMED or UNREACHABLE)	Manager
GET	/api/keys/:projectId/wrap-key	(Production only) The capsule's public wrapping key, used by the device to protect the CEK when sending it	wallet
POST	/api/keys/:pieceId	Registers the wrapped CEK (at registration)	wallet (registrant)
POST	/api/keys/:pieceId/release	Evaluates the decryption conditions, reconciles the database with the CIP-25 metadata (Output A1, Section 4.3), and returns the CEK if the conditions are met (prototype: TLS	wallet and signature

Method	Path	Description	Authentication
		only; production: re-wrapped with the device's public key). Used at opening	
POST	/api/keys/:projectId/revoke	(Production only) Revokes the KEK (crypto-shredding)	GROUP_OWNER
GET	/api/capsules/:id/verify	On-chain verification (existing). The result of the manifest_hash comparison is added in production	public

Table 6-1 Storage and Key APIs

7. Key Design Decisions and Their Rationale

This chapter summarizes the main design decisions made in this document and their rationale.

Decision	Content	Rationale
Encryption method	AES-256-GCM with the Web Crypto API, performed on the device	Authenticated encryption detects tampering during decryption; plaintext never leaves the device.
Keys	A random CEK per piece, stored wrapped; no derivation from the opening date/time	Keys derived from public information could be computed by anyone. Wrapped storage allows the key hierarchy to be strengthened in stages.
Deletion of personal data	Destroying the KEK makes the ciphertext undecryptable (crypto-shredding)	Ciphertext left in decentralized storage can effectively be treated as deleted, supporting deletion requests.
Group dissolution	Making sealed capsules undecryptable through KEK revocation is offered as an option	Allows the handling of sealed capsules to be decided according to the group's operating rules.
Long-term operation and service discontinuation	Escrow distribution of KEKs with threshold secret sharing, and publication of an offline decryption tool	Capsules can be opened by the managers' agreement even if the service disappears.
Cost control for large	Size limits by content type, linked to	Keeps storage costs

Decision	Content	Rationale
content	service plans (Table 2-4)	predictable as images and files are enabled.
Readiness for post-quantum cryptography	The enc.wrap enumeration can be extended, leaving room to add ML-KEM to the KEK layer	Prepares for storage periods of up to 100 years.

Table 7-1 Key Design Decisions and Their Rationale

8. Summary

This document defines, as Milestone 2 Output A2, the data storage design pattern for the service. For encryption and key management, content is encrypted with AES-256-GCM on the user's device, the ciphertext is bound to its capsule and piece through AAD, and integrity is checked in two stages using hashes of the ciphertext and the plaintext. Keys are generated randomly per piece and stored wrapped in a three-layer hierarchy in production, which the prototype simplifies to two layers. For redundancy and failover, the design provides three levels of redundancy for storage, retrieval paths, and location information, together with sequential gateway switching with hash verification, responses to six failure scenarios, and retention periods for backups; the prototype implements only hash verification, switching between two gateways, and standard database backups. For display and search, display criteria are defined by capsule state and viewer position, along with three search paths and a procedure for rebuilding the capsule list without the database. In response to the Milestone 1 review, pinning was unified on Blockfrost IPFS, operating rules and cost estimates were defined with the free Starter plan as the default, and the policy for migrating to Arweave was organized. Finally, the differences between the prototype and production implementations are shown side by side (Table 0-3). The status of each acceptance criterion is shown in Table 8-1.

Acceptance Criterion	How It Is Addressed	Reference
A2-1 Encryption and decryption	AES-256-GCM (Web Crypto), prevention	Chapter 3,

Acceptance Criterion	How It Is Addressed	Reference
design and key management method	of reuse through AAD, two-stage hash verification, the CEK, KEK, and MK key hierarchy (two layers in the prototype), procedures at registration and opening, and revocation, rotation, and escrow distribution	Figures 3-1 and 3-2
A2-2 Redundancy and failover strategy	Three levels of redundancy (storage, retrieval paths, location information), sequential gateway switching with hash verification, six failure scenarios and responses, and backup retention periods, with the prototype's scope stated	Chapter 4, Figure 4-1
A2-3 Display criteria for stored data and search mechanisms	Display criteria by state and position, three search paths (database, on-chain, storage side), order of retrieval, and re-indexing without the database	Chapter 5
(Reference) Milestone 1 review comment	Pinning unified on Blockfrost IPFS, correspondence of pin states, operating rules, plan selection (Starter by default, Hobby if needed) with capacity and request estimates, and the Arweave migration policy	Sections 0.1, 2.2, 2.5, and 2.6

Table 8-1 Status of Response to the Acceptance Criteria